

LAPORAN PRAKTIKUM
JOB SHEET 03 - ENKAPSULASI



Disusun Oleh:

Nama : Mohammad Hafidz Rafi' Rabbani
NIM : 254107020084
Kelas : 2H
No. Absen : 14

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2026

Daftar Isi

1. Tujuan	3
2. Praktikum	3
2.1. Percobaan 1: Enkapsulasi	3
2.2. Percobaan 2: Access Modifier	4
2.2.1. Pertanyaan	5
2.3. Percobaan 3: Getter dan Setter	6
2.4. Percobaan 4: Konstruktor dan Instansiasi	7
2.4.1. Pertanyaan	8
3. Tugas	10
3.1. Tugas 1, 2, dan 3: Enkapsulasi Nilai Usia (EncapDemo & EncapTest)	10
3.2. Tugas 4, 5, dan 6: Manajemen Kargo Ekspedisi (Kontainer & TestLogistik)	11
3.3. Tugas 7: Pemesanan Tiket Bioskop Aman (Tiket & TestBioskop)	12

1. Tujuan

- Memahami konsep constructor dan aturan pembuatannya pada bahasa pemrograman Java.
- Mengimplementasikan 4 jenis access modifier (public, protected, default, dan private).
- Memahami konsep attribute dan method pada class serta mekanisme instansiasinya.
- Mengimplementasikan setter dan getter untuk menerapkan enkapsulasi data.
- Membaca dan memetakan notasi UML Class Diagram ke dalam source code Java.

2. Praktikum

2.1. Percobaan 1: Enkapsulasi

Pada percobaan pertama, class `Motor` dibuat dengan attribute `kecepatan` dan `kontakOn` yang masih bertipe `public`. Dampaknya, class driver `MotorDemo` bisa langsung mengubah nilai `kecepatan` dari luar tanpa validasi kondisi mesin, sehingga kecepatan motor bisa tiba-tiba melonjak ke 50 saat kontak masih Off.

```
// MotorDemo.java
Motor motor = new Motor();
motor.printStatus();
motor.kecepatan = 50;
motor.printStatus();
```

```
● rafi@santaro experiments % java ./MotorDemo.java
Kontak Off
Kecepatan : 0

Kontak Off
Kecepatan : 50

○ rafi@santaro experiments % |
```

2.2. Percobaan 2: Access Modifier

Percobaan kedua memperbaiki class `Motor` dengan menerapkan enkapsulasi. Attribute `kecepatan` dan `kontakOn` diubah menjadi `private`, lalu aksesnya dikontrol lewat method `nyalakanMesin()`, `matikanMesin()`, `tambahKecepatan()`, dan `kurangiKecepatan()`. Ditambahkan juga batas maksimal kecepatan sebesar 100.

```
// Motor.java
public void tambahKecepatan() {
    if (kontakOn && kecepatan < maxKecepatan) {
        kecepatan += 5;
    } else if (kontakOn && kecepatan >= maxKecepatan) {
        System.out.println("Kecepatan tidak bisa bertambah karena kecepatan sudah mencapai batas maksimal\n");
    } else {
        System.out.println("Kecepatan tidak bisa bertambah karena mesin off!!\n");
    }
}
```

```
● rafi@santaro experiments % java ./MotorDemo.java
Kontak Off
Kecepatan : 0

Kecepatan tidak bisa bertambah karena mesin off!!

Kontak On
Kecepatan : 0

Kontak On
Kecepatan : 5

Kontak On
Kecepatan : 10

Kontak On
Kecepatan : 15

Kontak Off
Kecepatan : 0

○ rafi@santaro experiments %
```

2.2.1. Pertanyaan

1. Pada class MotorDemo, saat kita menambah kecepatan untuk pertama kalinya, mengapa muncul peringatan “Kecepatan tidak bisa bertambah karena Mesin Off!”?
 - Karena method `tambahKecepatan()` melakukan validasi apakah attribute `kontakOn` bernilai `true`. Kalau nilainya masih `false` (mesin off), penambahan kecepatan tidak dijalankan dan program menampilkan pesan peringatan tersebut.
2. Mengapa atribut kecepatan dan kontakOn diset private?
 - Untuk menyesuaikan dengan kondisi di dunia nyata (IRL). Kecepatan motor tidak masuk akal jika bisa diubah bebas dari luar tanpa menyalakan mesin. Dengan di-set `private`, kedua attribute tersebut tidak bisa diakses langsung dan pengubahannya harus lewat method yang sudah memiliki validasi.
3. Ubah class Motor sehingga kecepatan maksimalnya adalah 100! Jelaskan modifikasi logika yang dilakukan.
 - Menambahkan attribute `maxKecepatan = 100` dan validasi pada method `tambahKecepatan()`. Kecepatan hanya bertambah jika `kontakOn` bernilai `true` dan nilai `kecepatan` saat ini masih di bawah 100. Jika sudah mencapai 100 atau lebih, program menampilkan pemberitahuan bahwa batas maksimal sudah tercapai.

2.3. Percobaan 3: Getter dan Setter

Pada percobaan ketiga, dibuat class `Anggota` dengan attribute `nama`, `alamat`, dan `simpanan` yang di-set `private`. Attribute `nama` dan `alamat` diberi method getter dan setter, sedangkan `simpanan` hanya memiliki getter dan nilainya diubah lewat method transaksi `setor()` dan `pinjam()`.

```
// Anggota.java
public void setor(float uang) {
    simpanan += uang;
}
public void pinjam(float uang) {
    simpanan -= uang;
}
```

```
● rafi@santaro experiments % java ./KoperasiDemo.java
Simpanan Iwan : Rp 100000.0
Pinjaman Iwan : Rp 95000.0
○ rafi@santaro experiments %
```

2.4. Percobaan 4: Konstruktor dan Instansiasi

Pada percobaan keempat, ditambahkan constructor dengan parameter `Anggota(String nama, String alamat)` pada class `Anggota`. Constructor ini memastikan attribute identitas langsung terisi nilai awal saat objek dibuat dengan `new`, dan nilai saldo `simpanan` otomatis dimulai dari 0.

```
// Anggota.java
public Anggota(String nama, String alamat) {
    this.nama = nama;
    this.alamat = alamat;
    this.simpanan = 0;
}
```

```
● rafi@santaro experiments % java ./KoperasiDemo.java
Simpanan Iwan : Rp 0.0
Simpanan Iwan : Rp 100000.0
Pinjaman Iwan : Rp 95000.0
○ rafi@santaro experiments % |
```

2.4.1. Pertanyaan

1. Apa yang dimaksud dengan getter dan setter?
 - Getter dan setter adalah method khusus yang digunakan untuk mengakses dan mengubah private attribute di class Java. Getter digunakan untuk mengambil atau membaca nilai private attribute, sedangkan setter untuk mengatur atau memperbarui nilainya.
2. Apa kegunaan dari method getSimpanan()?
 - Fungsinya untuk mengambil dan membaca nilai dari private attribute `simpanan`.
3. Method apa yang digunakan untuk menambah saldo pada class Anggota?
 - Method `setor()` yang memiliki parameter `uang`, yang nilainya kemudian ditambahkan ke attribute `simpanan`.
4. Apa yang dimaksud dengan konstruktor?
 - Constructor adalah method spesial di Java yang otomatis dieksekusi saat menginisialisasi object baru menggunakan keyword `new`. Biasanya digunakan untuk memberikan nilai default awal pada attribute object agar tidak kosong saat dibuat. Nama constructor harus sama dengan nama class dan tidak memiliki return type.
5. Sebutkan aturan-aturan dalam membuat konstruktor di Java!
 - (1) Nama constructor harus sama persis dengan nama class. (2) Tidak memiliki return type apapun, termasuk `void`. (3) Boleh menggunakan access modifier (public, private, protected, default), namun tidak boleh menggunakan modifier seperti abstract, static, final, atau synchronized.
6. Apakah boleh sebuah konstruktor bertipe (memiliki access modifier) private? Jelaskan kegunaannya!
 - Boleh. Digunakan agar tidak ada pembuatan object dari luar class tersebut. Biasanya diterapkan untuk utility class atau helper class yang hanya berisi method static.
7. Kapan kita harus menggunakan konstruktor dengan passing parameter?

- Ketika kita ingin memberikan nilai awal yang spesifik pada suatu attribute saat object pertama kali dibuat, sehingga mencegah attribute tidak memiliki nilai yang valid tepat setelah diinisialisasi.
8. Apa perbedaan antara inisialisasi atribut dan instansiasi atribut/objek?
- Instantiation adalah proses membuat object baru di memory heap menggunakan keyword `new`. Sedangkan initialization adalah proses mengisi atau memberikan nilai awal ke suatu attribute atau variable.
9. Apa perbedaan antara inisialisasi method (definisi method) dan instansiasi / pemanggilan method?
- Secara konsep Java murni tidak ada istilah instansiasi method, karena yang di-instantiate ke heap memory adalah class/object. Namun dalam konteks praktikum: inisialisasi method adalah saat kita membuat struktur method di dalam class (nama, parameter, return type, body kode), sedangkan pemanggilan method (method invocation) adalah saat method tersebut dieksekusi lewat object yang sudah dibuat (`object.method()`).

3. Tugas

3.1. Tugas 1, 2, dan 3: Enkapsulasi Nilai Usia (EncapDemo & EncapTest)

Class `EncapDemo` mengimplementasikan enkapsulasi untuk attribute `name` dan `age`. Method `setAge(int age)` dimodifikasi menggunakan validasi `if-else`: jika input `age` lebih dari 30 maka nilai otomatis di-set ke 30, dan jika kurang dari 18 otomatis di-set ke 18.

```
// EncapDemo.java
public void setAge(int age) {
    if (age > 30) {
        this.age = 30;
    } else if (age < 18) {
        this.age = 18;
    } else {
        this.age = age;
    }
}
```

Pada `EncapTest`, ketika nilai `age` diisi 35 lewat `encap.setAge(35)`, output yang tampil di konsol tetap 30 karena terjaring oleh validasi `age > 30`.

```
● rafi@santaro src % java ./assignments/EncapTest.java
  Name      : james
  Age       : 30
○ rafi@santaro src %
```

3.2. Tugas 4, 5, dan 6: Manajemen Kargo Ekspedisi (Kontainer & TestLogistik)

Membuat class `Kontainer` dengan semua attribute di-set `private` (`nomorResi`, `namaPemilik`, `kapasitasMaksimal`, `beratMuatanSaatIni`). Pada constructor, data resi, pemilik, dan kapasitas maksimal diinisialisasi dengan berat awal 0. Method `tambahMuatan()` memvalidasi agar tidak melebihi kapasitas maksimal, sedangkan `turunkanMuatan()` dibatasi maksimal 50% dari berat muatan saat ini demi alasan keselamatan kerja crane. Pada class driver `TestLogistik`, ditambahkan interaksi input dinamis menggunakan `java.util.Scanner`.

```
// Kontainer.java
public void turunkanMuatan(float jumlah) {
    if (jumlah > beratMuatanSaatIni / 2) {
        System.out.println("Maaf, demi keselamatan, pembongkaran muatan satu kali jalan tidak boleh melebihi 50% dari muatan saat ini!");
    } else {
        if (beratMuatanSaatIni - jumlah >= 0) {
            beratMuatanSaatIni -= jumlah;
        } else {
            System.out.println("Jumlah yang diturunkan melebihi jumlah yang ada.");
        }
    }
}
```

Saat diuji coba memasukkan muatan 6.000 kg, sistem menolak karena kapasitas maksimal kontainer hanya 5.000 kg.

```
● rafi@santaro src % java ./assignments/TestLogistik.java
Nama Pemilik Kontainer: PT. Maju Bersama
Kapasitas Maksimal: 5000.0 kg

Memasukkan muatan baru seberat 6.000 kg...
Maaf, berat muatan melebihi kapasitas maksimal kontainer.
Berat muatan saat ini: 0.0 kg

Memasukkan muatan baru seberat 4.000 kg...
Berat muatan saat ini: 4000.0 kg

Membongkar muatan seberat 500 kg...
Berat muatan saat ini: 3500.0 kg

Membongkar muatan seberat 1.500 kg...
Berat muatan saat ini: 2000.0 kg
○ rafi@santaro src % |
```

3.3. Tugas 7: Pemesanan Tiket Bioskop Aman (Tiket & TestBioskop)

Membuat class `Tiket` dengan attribute `judulFilm`, `hargaDasar`, dan `statusPembayaran` yang di-set `private`. Pada constructor, nilai `statusPembayaran` default-nya selalu `false` (belum lunas), dan jika input `hargaDasar` negatif (< 0) otomatis fallback ke default Rp 35.000,00. Attribute `statusPembayaran` bersifat read-only tanpa method setter, dan hanya bisa diubah menjadi `true` lewat method `lakukanPembayaran()`.

```
// Tiket.java
public Tiket(String judulFilm, Double hargaDasar) {
    this.judulFilm = judulFilm;
    if (hargaDasar < 0) {
        this.hargaDasar = 35000.0;
    } else {
        this.hargaDasar = hargaDasar;
    }
    this.statusPembayaran = false;
}

public void lakukanPembayaran() {
    this.statusPembayaran = true;
}
```

```
● rafi@santaro src % java ./assignments/TestBioskop.java
Film: avengers: endgame
Harga tiket: 35000.0
Status Lunas? false

Memproses pembayaran....
Status Lunas? true
○ rafi@santaro src % |
```